Number Series Program

1)WAJP to Print Fibonacci Series.

The **Fibonacci Series** is a sequence in which each number is the sum of the two preceding ones.

```
public class fibo {
public static void main(String[] args) {
    int a=0;
    int b=1;
    System.out.println(a);
    System.out.println(b);
    int z=1;
    while (z<10) {
        int c=a+b;
        a=b;
        b=c;
        z++;
        System.out.println(c);
    }
}
}
```

2)WAJP to print even no from 1 to 100 using recursion.

```
public class ToPrintEvenNumberUsingRecursion
{
public static void main(String[] args) {
    even(1);
}
public static void even(int a)
{
int b=100;
if (a%2==0) {
    System.out.println(a);
}
if (a>=10) {
    return ;
}
a++;
even(a);
}
}
```

3)WAJP to Print odd no from 1 to 100 using recursion.

```
public class ToPrintOddNumberUsingRecursion {
public static void main(String[] args) {
    odd(1);
}
public static void odd(int a)
{
int b=100;
if (a%2!=0) {
    System.out.println(a);
}
if (a>=10) {
    return ;
}
a++;
odd(a);
}
}
```

4)WAJP to Check the User entered number is prime or not

Prime Number: A prime number is **a whole number greater than 1 whose only factors are 1 and itself**

Ex:2,3,5,7,13,….n

```
import java.util.Scanner;

public class DesignAMethodToReturnPrimeNUmber
{
public static void main(String[] args) {
Scanner s = new Scanner(System.in);
boolean res=prime(8);
System.out.println(res);
if (res) {
    System.out.println("Prime Number");
} else {
    System.out.println("Not a Prime Number");
}
}
public static boolean prime(int n)
{
    int i=2;
    int count=0;
    while (i<=n/2) {
        if (n%i==0) {
            return false;
        }
        i++;
    }
```

```
    return true;
}}
```

5)WAJP to find the factorial of the number

Factorial of a positive integer (number) is the sum of multiplication of all the integers smaller than that positive integer
Ex: 4 , 4*3*2*1=24

```
public class Factorial {
public static void main(String[] args) {
    int a=4;
    int res=fact(a);
    System.out.println(res);
}

private static int fact(int a) {
    int fact=1;
    while (a>1) {
        fact=fact*a;
        a--;
    }
    return fact;
}
}
```

6)WAJP to Check User Entered number is special two Digit Number or not.

A special number can be defined as a number in which the sum of the product of its digits and the sum of the digits is equal to the number.

Number: 59
Digits are: 5 and 9

Sum of the digits (SoD) = 5 + 9 = 14
Product of the digits (PoD) = 5 X 9 = 45
Sum of the SoD and PoD = 14 + 45 = 59

```
import java.util.Scanner;

public class SpecialTwoDigitNumber {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter the number");
    int n=s.nextInt();
     int a=n%10;
     int b=n/10;
     int res=(a*b)+(a+b);
     if (n==res) {
       System.out.println("Special Two Digit
Number");
```

```
    } else {
        System.out.println("It is not a
Special Two Digit number");
    }

}
}
```

7)WAJP to design a method to find sum of first n natural number.

```
public class SumofNumber {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter the Starting
Range");
    int n=s.nextInt();
    System.out.println("Enter the Last
Range");
    int l=s.nextInt();
    int sum=0;
    while (n<=l) {
        sum=sum+n;
        n++;
    }
    System.out.println(sum);
}
}
```

8)Find the SquareRoot of Number

```
public class SquareRoot {
public static void main(String[] args) {
    int num=64;
    int count=0;
    int i=1;
    while (i<=num/2) {
        int sq=i*i;
        if (sq==num) {
            count++;
            break;
        }
        i++;
    }
    if (count==1) {
        System.out.println("Square root
of"+num+"="+i);
    }
    else
    {
        System.out.println(num+"is not a
Square no");
    }
}
}
```

9)WAJP to find factors of given number

Factors of a number are defined as numbers that divide the original number evenly or exactly.

```
public class ToFindTheFactor {
public static void main(String[] args) {
    int num=60;
    int i=1;
    int count=0;

    while (i<=num) {
        if (num%i==0) {
            System.out.println(i);
            count++;
        }
        i++;
    }
    System.out.println("Total No of Factors
"+count);
}
}
```

10)WAJP to reverse a Number

Input:2861
Output:1682

```java
import java.util.Scanner;

public class ToReverseNumber {
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n=sc.nextInt();
    while (n>0) {
        int rem=n%10;
        System.out.print(rem);
        n=n/10;
    }
}
}
```

11)WAJP to reverse a Number and Store it a Container.

```java
import java.util.Scanner;

public class
ToReverseANumberAndStoreItInAContainer {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
    int rev=0;
    while (n>0) {
        int rem=n%10;
        rev=rev*10+rem;
        n=n/10;
    }
    System.out.println(rev);
}
}
```

12)WAJP to check the given number is palindrome or not.

A palindrome number is a number that remains the same when digits are reversed

Ex : 121

```
import java.util.Scanner;

public class Palindrome {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
    int rev=0;
    int temp=n;
    while (n>0) {
        int rem=n%10;
        rev=rev*10+rem;
        n=n/10;
    }
    if (temp==rev) {
        System.out.println("The Number is
Palindrome");
    } else {
         System.out.println("The Number is Not
Palindrome");
    }
```

```
}
}
```

13)WAJP to Count no of Digit in the Number.

Input:1537

Output:4

```
import java.util.Scanner;

public class ToCountTheDigit {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
    int count=0;
    while (n>0) {
        int rem=n%10;
        count++;
        n=n/10;
    }
    System.out.println(count);
}
}
```

14)WAJP to Find nth Power of the number

Ex:

$2^3 = 8;$

```java
import java.util.Scanner;

public class ToFindNthPower {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter the Number");
    int n=s.nextInt();
    System.out.println("Enter the power");
    int power=s.nextInt();
    int prod=1;
    while (power>0) {
        prod=prod*n;
        power--;
    }
    System.out.println(prod);
}
}
```

15)Design a Method to Check Given Number is Prime or Not.

```
import java.util.Scanner;

public class DesignAMethodToReturnPrimeNUmber
{
public static void main(String[] args) {
Scanner s = new Scanner(System.in);
boolean res=prime(8);
System.out.println(res);
if (res) {
    System.out.println("Prime Number");
} else {
    System.out.println("Not a Prime Number");
}
}
public static boolean prime(int n)
{
    int i=2;
    int count=0;
    while (i<=n/2) {
        if (n%i==0) {
            return false;
        }
        i++;
    }
    return true;
```

```
}
}
```

16)WAJP to check given number is Strong number

A strong number is a number whose sum of each digit's factorial is the number itself

Ex:

Number =145

1!+4!+5!=145

```
import java.util.Scanner;

public class DesignAMethodToFindStrongNumber
{
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
    String res=strong(n);
    System.out.println(res);
}

private static String strong(int n) {
    int sum=0;
    int temp=n;
    while (n>0) {
        int rem=n%10;
        int fact=1;
```

```
        while (rem>0) {
            fact=fact*rem;
            rem--;
        }
        sum=sum+fact;
        n=n/10;
    }
    if (temp==sum) {
        return "Strong Number";
    }
    return "Not a Strong Number";
}
}
```

17)WAJP to Check the number is Spy number

A positive integer is called a spy number if the sum and product of its digits are equal.

Ex:123

1+2+3==1*2*3

```
import java.util.Scanner;

public class DesingAMethodToFindSpyNumber {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
    String res=spy(n);
    System.out.println(res);
}

private static String spy(int n) {
    int sum=0;
    int prod=1;
    while (n>0) {
        int rem=n%10;
        sum=sum+rem;
        prod=prod*rem;
        n=n/10;
    }
```

```
    if (sum==prod) {
        return "Spy Number";
    }
    return "Not a Spy Number";
}
}
```

18)WAJP to check the number is neon number

A positive integer whose sum of digits of its square is equal to the number itself is called a neon number

Ex:

Number : 9

Square:9*9=81

Sum of digit:1+8==9

```
public class DesignAMethodToFindNeonNumber {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    System.out.println("Enter the Number");
    int n=s.nextInt();
    String res=neon(n);
    System.out.println(res);
}
public static String neon(int n)
{
int sq=n*n;
int sum=0;
while (sq>0) {
    int rem=sq%10;
```

```
    sum=sum+rem;
    sq=sq/10;
}
if (n==sum) {
    return "Neon Number";
}
return "Not a Neon Number";
}
}
```

19)WAJP to check the number is perfect number

perfect number, a positive integer that is equal to the sum of its proper divisors.

Ex:number =6

Divisor=1,2,3

Sum of the divisor=1+2+3==6

```
import java.util.Scanner;

public class DesignAMethodToFindPerfectNumber
{
public static void main(String[] args) {
    Scanner s=new Scanner(System.in);
    System.out.println("Enter the Number :
");
     int n=s.nextInt();
     String res=perfect(n);
     System.out.println(res);
}

private static String perfect(int n) {
    int sum=0;
    int i=1;
    while (i<=n/2) {
```

```
            if (n%i==0) {
                sum=sum+i;
            }
            i++;
        }
        if (sum==n) {
            return "Perfect Number";
        }
        return "Not a Perfect Number";
    }
}
```

20)WAJP to check the number is ArmStrong Number

The number in any given number base, which forms the total of the same number, when each of its digits is raised to the power of the number of digits in the number.

Ex:Number :153

Number of digit=3;

Power of each digit =3

Sum of Power of each Digit=1^3 + 5^3 +3^3=153

```
import java.util.Scanner;

public class
DesignAMethodToFindArmStrongNumber {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
    String res=arm(n);
    System.out.println(res);
}

private static String arm(int n) {
```

```
        int sum=0;
        int temp=n;
        int temp1=n;
        int count=0;
        while (n>0) {
            n=n/10;
            count++;
        }
        while (temp>0) {
            int rem=temp%10;
            int power=count;
            int prod=1;
            while (power>0) {
                prod=prod*rem;
                power--;

            }
            sum=sum+prod;
            temp=temp/10;
        }
        if (temp1==sum) {
            return "ArmStrong Number";
        }
        return "Not a ArmStrong Number";
    }
}
```

21)WAJP to check the given number is Sunny Number.

A number is called a sunny number if the number next to the given number is a perfect square.

Example:8 is Sunny Number next number 9(N+1)is perfect Square.

```
import java.util.Scanner;

public class DesignAMethodToFindSunnyNumber {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
    String res=sunny(n);
    System.out.println(res);
}

public static String sunny(int n) {
    int a=n+1;
    int i=1;
    int count=0;
    while (i<a) {
        int sq=i*i;
        if (sq==a) {
            count++;
```

```
            break;
        }
        i++;
    }
    if (count==1) {
        return "Sunny Number";
    }
    return "Not a Sunny Number";
}
}
```

22)WAJP to check given Number is Happy Number

A number is called happy if it leads to 1 after a sequence of steps wherein each step number is replaced by the sum of squares of its digit that is if we start with Happy Number and keep replacing it with digits square sum, we reach 1

Ex:

Input: n = 19

Output: Happy Number

19 is Happy Number,

1^2 + 9^2 = 82

8^2 + 2^2 = 68

6^2 + 8^2 = 100

1^2 + 0^2 + 0^2 = 1

As we reached to 1, 19 is a Happy Number.

```
import java.util.Scanner;

public class
DesignAMethodToCheckNumberIsHappyNumber {
public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n=s.nextInt();
```

```
    String res=happy(n);
    System.out.println(res);
}
public static String happy(int num)
{
    while (num>9) {
        int sum=0;
        while (num>0) {
            int rem=num%10;
            sum=sum+rem*rem;
            num=num/10;
        }
        num=sum;
    }
    if (num==1 || num==7) {
        return "Happy number";
    }
    return "Not a Happy Number";

}
}
```

23)WAJP to Convert Decimal to Binary

Input:10

Ouput:1010

```
public class DecimalToBinary {
public static void main(String[] args) {
    int num=10;
    String s=" ";
    do {
        int rem=num%2;
        s=rem+s;
        num=num/2;
    } while (num!=0);
    System.out.println(s);
}
}
```

24)WAJP to Convert decimal to HexaDecimal

Hexadecimal Number System is one the type of Number Representation techniques, in which there value of base is 16. That means there are only 16 symbols or possible digit values, there are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F. Where A, B, C, D, E and F are single bit representations of decimal value 10, 11, 12, 13, 14 and 15 respectively.

```
public class DecimalToHexaDecimal {
public static void main(String[] args) {
int num=11;
String s=" ";
do {
    int rem=num%16;
    if (rem<10) {
        s=rem+s;
    } else {
       s=(char)(rem+55)+s;
       num=num/16;
    }
} while (num!=0);
    System.out.println(s);
}
}
```

25)WAJP to Convert Decimal to Octal

Octal Number System is a type of number system that has a base of eight and uses digits from 0 to 7

```
public class DecimalToOctal {
public static void main(String[] args) {
    int num=8;
    String s=" ";
    do {
        int rem=num%8;
        s=rem+s;
        num=num/8;
    } while (num!=0);
    System.out.println(s);
}
}
```

26)WAJP to Print Prime number between 1 to 1000

```
public class Prime1to1000 {

    public static void main(String[] args) {
        for(int j=0;j<=1000;j++)
        {
              int n=j;
              int count=0;
              int i=2;


              while(i<n/2)
              {
                 if(n%i==0)
                 {
                     count++;
                     break;
                 }
                 i++;
              }
              if(count==0)
              {
                 System.out.println(n);

              }
        }
    }

  }
```